Examination Performance Analysis

Python Programming Case Study

1. Problem Statement

Teachers need a simple way to evaluate how students perform on individual exam questions. By comparing students' answers with an answer key, the program can calculate the success rate for each question, classify question difficulty, and identify the question missed most frequently. This helps teachers recognize topics that may need additional explanation.

2. Objective

To create a Python program that compares multiple students' answers with a correct answer key, calculates the percentage of correct responses for each question, labels performance as Easy, Moderate, or Difficult, and finds the question with the greatest number of incorrect responses.

3. Assumptions

• The answer key contains five questions, with one correct option per question.

• Each student's response list contains answers in the same order as the answer key.

• All students have attempted all five questions.

• A response is correct when it exactly matches the corresponding answer-key value.

• A success rate above 70% is labelled Easy; a rate above 40% and up to 70% is Moderate; otherwise it is Difficult.

• The question with the largest number of incorrect responses is considered the most frequently missed question.

4. Algorithm

1. Start the program.

2. Define the correct answer key and the answer lists for all students.

3. Initialize a success counter for each question to zero.

4. For every student, compare each answer with the matching answer-key entry.

5. When an answer is correct, increment that question's success counter.

6. For each question, calculate success rate as (correct responses ÷ number of students) × 100.

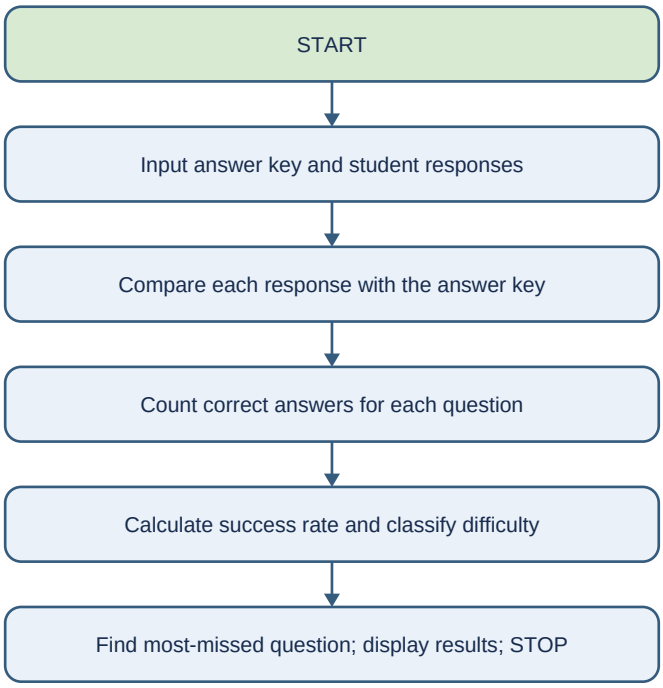
7. Display the question number and success rate.

8. Classify performance as Easy, Moderate, or Difficult using the defined thresholds.

9. Find the question with the smallest success count, which corresponds to the most frequently missed question.

10. Display the result and stop.

5. Flowchart



Repeat comparisons for every student and every question.

6. Pseudocode

```
BEGIN
    answer_key ← ["A", "B", "C", "D", "A"]
    students ← list of students' answers
    success ← [0, 0, 0, 0, 0]

    FOR each student in students
        FOR each question index i
            IF student[i] = answer_key[i]
                success[i] ← success[i] + 1
        END FOR
    END FOR

    FOR each question index i
        rate ← (success[i] / number of students) × 100
        PRINT question number and rate

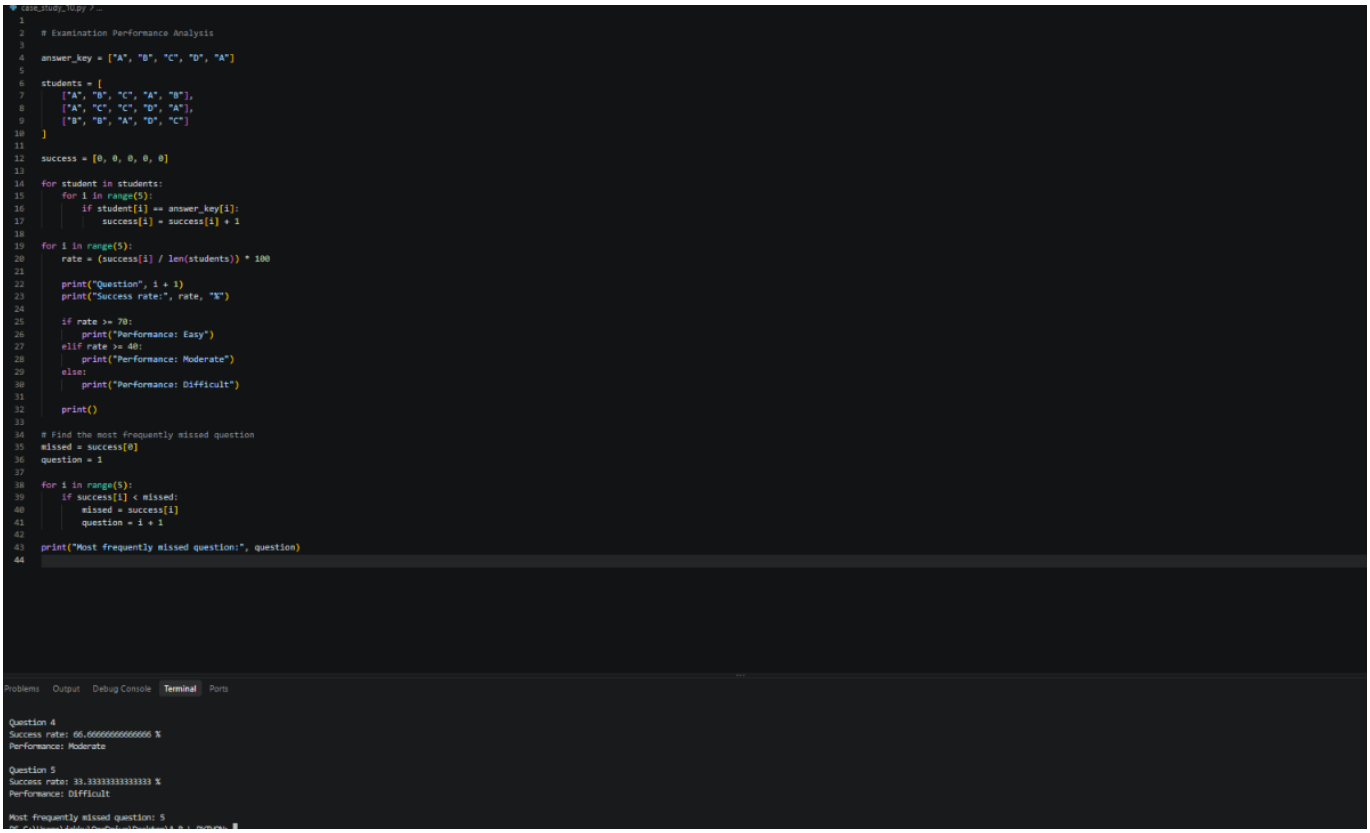
        IF rate > 70
            PRINT "Performance: Easy"
        ELSE IF rate > 40
            PRINT "Performance: Moderate"
        ELSE
            PRINT "Performance: Difficult"
    END FOR

    missed ← success[0]
    question ← 1
    FOR each question index i
        IF success[i] < missed
            missed ← success[i]
            question ← i + 1
    END FOR

    PRINT most frequently missed question
END
```

7. Source Code and Output Screenshot

The screenshot below contains the provided Python source code and terminal output.



8. Conclusion

The Examination Performance Analysis program demonstrates how Python can support question-wise assessment. It uses nested loops and comparisons to count correct answers, calculates success rates, categorizes question difficulty, and identifies the question with the lowest success count. For the supplied sample, Question 5 is reported as the most frequently missed question. The results can help educators focus revision and provide additional support where students need it most.